



Taller 2: Abstracción de datos y sintáxis abstracta

Fundamentos de Interpretación y Compilación de LP / Grupo 01 / Prof. Robinson Duque / Emily Nuñez/ 2026-II

Los indicadores de logro que se abordarán en este taller están determinados por:

- Implementa Tipos Abstractos de Datos utilizando dos componentes: una interfaz y una implementación
- Utiliza estrategias con listas y datatypes para representar tipos de dato definidos por una gramática
- Construye un árbol de sintaxis abstracta a partir de una representación concreta en una gramática BNF
- Implementa procedimientos Parse y Unparse para derivar entre representación concreta y abstracta de una gramática BNF
- Utiliza herramientas para especificar analizadores léxicos y sintacticos de una gramática BNF

1 Expresiones aritméticas

En los lenguajes de programación es necesario definir una forma precisa de representar las expresiones que pueden ser escritas por un usuario. Posteriormente, un intérprete o compilador puede transformar estas expresiones en una representación interna que facilite su procesamiento. En este taller se trabajará con un pequeño lenguaje de expresiones aritméticas llamado EXPRESION.

El lenguaje utilizará **notación infija**. Esto significa que el operador aparece en medio de sus operandos. Por ejemplo, la siguiente expresión representa la suma entre el número 3 y la variable x :

$(3 + x)$

El objetivo del taller es construir progresivamente los componentes necesarios para procesar este pequeño lenguaje. En particular, se desarrollarán los siguientes componentes:

1. un Tipo Abstracto de Datos para representar expresiones y una gramática para definir la sintaxis concreta;
2. una representación abstracta de las expresiones;
3. funciones **PARSE** para transformar la representación concreta en representación abstracta, tanto para la implementación basada en **listas** como para la implementación basada en **datatypes**;
4. funciones **UNPARSE** para realizar la transformación inversa, tanto para la implementación basada en **listas** como para la implementación basada en **datatypes**;
5. una función para evaluar las expresiones que incluye un TAD para representar ambientes

2 (30 pts) Tipo Abstracto de Datos para Expresiones

Se desea construir un Tipo Abstracto de Datos denominado **EXPRESION**, que permita representar expresiones aritméticas. Una expresión puede ser:

- una constante entera;
- una variable;
- una suma de dos expresiones;
- una resta de dos expresiones;
- una multiplicación de dos expresiones.

Formalmente, se puede definir la estructura conceptual del TAD mediante las siguientes construcciones:

```

EXPRESION ::= CONST(numero)
           | VAR(variable)
           | ADD(EXPRESION, EXPRESION)
           | SUB(EXPRESION, EXPRESION)
           | MUL(EXPRESION, EXPRESION)

```

Por ejemplo, las siguientes expresiones en *sintaxis concreta* pertenecen al lenguaje:

```

5
x
(3 + x)
(y - 2)
((3 + x) * (y - 2))

```

La expresión:

```
((3 + x) * (y - 2))
```

puede representarse conceptualmente en *sintaxis abstracta* mediante listas así:

```

(mul
  (add (const 3) (var x))
  (sub (var y) (const 2)))

```

(15 pts) Implemente el TAD utilizando **listas** como mecanismo de representación interna. La implementación deberá incluir los constructores, extractores y predicados necesarios. Como mínimo deberá existir funcionalidad equivalente a:

```
const          ;Constructor de constantes
const?         ;Predicado
const->value    ;Extractor
```

```
var            ;Constructor de variables
var?           ;Predicado
var->name       ;Extractor
```

```
add
add?
add->left
add->right
```

```
sub
sub?
sub->left
sub->right
```

```
mul
mul?
mul->left
mul->right
```

```
; puede construir un predicado para cada regla en la gramática
; o un predicado para toda la expresion llamado EXPRESION?
```

La representación interna utilizada por las listas es decisión del estudiante, siempre que se respeten las operaciones definidas por el TAD. Incluya ejemplos que demuestren:

1. la creación de constantes;
2. la creación de variables;

3. la creación de sumas;
4. la creación de restas;
5. la creación de multiplicaciones;
6. la construcción de expresiones anidadas.
7. la construcción de al menos dos expresiones no triviales.

(15 pts) Implemente nuevamente el TAD utilizando `datatype`. La implementación deberá proporcionar una funcionalidad equivalente a la implementación basada en listas. Construya al menos tres expresiones no triviales utilizando esta representación.

3 (40pts) Funciones Parse y Unparse

Una vez definida la representación concreta del lenguaje y el TAD de expresiones, se requiere construir funciones que permitan transformar una representación en otra.

- **(10 pts)** Construya una función:

`(PARSE-LT expr)`

que reciba una lista correspondiente a una expresión concreta y construya la representación abstracta de la expresión basada en listas. Por ejemplo:

`(PARSE-LT '( (3 + x) * (y - 2)))`

deberá producir una representación basada en **listas**.

```
(mul
  (add (const 3) (var x))
  (sub (var y) (const 2)))
```

Incluya por lo menos cinco ejemplos de utilización de `PARSE-LT`.

- (10 pts) Construya una función:

```
(PARSE-DT expr)
```

que reciba una lista correspondiente a una expresión concreta y construya la representación abstracta de la expresión basada en datatypes. Por ejemplo:

```
(PARSE-DT '(3 + x))
```

deberá producir una representación basada en el **struct** que maneja internamente **define-datatype**. Incluya por lo menos cinco ejemplos de utilización de PARSE-DT.

- (10 pts) Construya una función:

```
(UNPARSE-LT expr)
```

que reciba una representación abstracta de una expresión basada en **listas** y produzca su correspondiente **representación concreta**, también representada mediante listas de Racket. Por ejemplo:

```
(UNPARSE-LT
  '(mul
    (add (const 3) (var x))
    (sub (var y) (const 2))))
```

deberá producir una sintaxis concreta equivalente a:

```
'( (3 + x) * (y - 2))
```

Incluya al menos cinco ejemplos de utilización de UNPARSE. Además, demuestre mediante ejemplos que, para expresiones válidas:

```
(UNPARSE-LT (PARSE-LT expr))
```

la expresión resultante deberá representar la misma estructura sintáctica que la expresión concreta original.

- (10 pts) Construya una función:

`(UNPARSE-DT expr)`

que reciba una representación abstracta de una expresión basada en **datatypes** y produzca su representación concreta basada en listas. Por ejemplo:

`(UNPARSE-DT (PARSE-DT '( (3 + x) * (y - 2)) ))`

produce una representación equivalente a la expresión concreta original basada en listas.

`'( (3 + x) * (y - 2))`

Incluya al menos cinco ejemplos de utilización de `UNPARSE-DT`.

4 (30 pts) Evaluación de Expresiones

En esta última etapa del taller se deberá implementar un evaluador para las expresiones definidas mediante la gramática del lenguaje. La función evaluadora recibirá una expresión en **sintaxis abstracta** y un ambiente, y deberá retornar el valor obtenido al evaluar dicha expresión. Cuando una expresión contiene variables, es necesario conocer el valor asociado a cada una de ellas. Para esto se utilizará el concepto de **ambiente** estudiado en clase. La función:

`(APPLY-ENV ambiente variable)`

deberá retornar el valor asociado a **variable** en el ambiente. La representación del ambiente será con **datatypes**. Adicionalmente, Se deberá implementar la función:

`(EVALUAR expr ambiente)`

donde **expr** corresponde a una expresión en sintaxis abstracta y **ambiente** corresponde al ambiente utilizado para resolver los valores de las variables. La función deberá retornar el resultado de evaluar **expr**.

Para implementar **EVALUAR**, se debe seguir la estructura definida por la **gramática del lenguaje**. Es decir, se debe analizar cada uno de los posibles tipos de expresiones definidos en la gramática y establecer cómo se evalúa cada uno de ellos. Por ejemplo, considere el siguiente ambiente:

```
(extend-env
 '(x y z)
 '(5 10 2)
 (empty-env))
```

En este ambiente se tiene:

- **x** asociado al valor 5,
- **y** asociado al valor 10,
- **z** asociado al valor 2.

Considere ahora la siguiente expresión escrita utilizando la notación infija del lenguaje:

```
'((3 * x) * (y - 2))
```

Esta expresión deberá ser convertida previamente a su correspondiente **sintaxis abstracta**, utilizando la representación de **datatypes**. Por ejemplo:

```
(PARSE-DT '((3 * x) * (y - 2)))
```

Finalmente, la evaluación se realizará mediante:

```
(EVALUAR
 (PARSE-DT '((3 * x) * (y - 2)))
 (extend-env
 '(x y z)
 '(5 10 2)
 (empty-env))
 )
```

y deberá producir como resultado:

$$120$$

pues, utilizando los valores definidos en el ambiente,

$$((3 * x) * (y - 2)) = ((3 * 5) * (10 - 2)) = (15 * 8) = 120.$$

Uso Ético de Inteligencia Artificial

Usos permitidos de IA: se permite utilizar herramientas de IA únicamente para:

1. Aclarar conceptos teóricos.
2. Solicitar explicaciones adicionales sobre el tema de estudio.
3. Pedir ejemplos ilustrativos distintos a los del taller.
4. Revisar estilo, redacción de documentación o comentarios.
5. Comprender mensajes de error del intérprete.

Usos NO permitidos de IA:

1. Solicitar a la IA la solución completa o parcial de los ejercicios.
2. Copiar y pegar código generado por IA sin comprensión.
3. Modificar mínimamente una solución generada por IA y presentarla como propia.
4. Usar IA durante una posible sustentación oral sin autorización.

Declaración obligatoria de uso de IA: si se utilizó IA en alguna parte del proceso, el grupo debe incluir al inicio del archivo:

;; Declaración de uso de IA:

;; Se utilizó IA para: (describir brevemente el uso)

;; Se declara que todos los integrantes comprenden y pueden explicar

;; completamente cada solución implementada.

Sustentación obligatoria: el Profesor podrá:

1. Solicitar sustentación individual o grupal de cualquier ejercicio.
2. Pedir modificación en vivo de una función.

3. Cambiar ejemplos de prueba.
4. Pedir reconstrucción parcial o total de algún algoritmo o funcionalidad.

Si un grupo/estudiante no puede explicar o defender su solución, la nota del ejercicio/taller será anulada (0.0), incluso si la solución es correcta y pasa las pruebas.

Criterios para detectar uso inadecuado de IA: Los siguientes criterios podrán considerarse señales de alerta; sin embargo, **no constituyen una lista exhaustiva ni limitativa**. El docente podrá apoyarse en otros elementos de juicio académico cuando lo considere pertinente:

1. Uso de estructuras o funciones no vistas en clase.
2. Estilos de programación inconsistentes con el nivel del curso.
3. Soluciones excesivamente complejas para el objetivo del ejercicio.
4. Código que los estudiantes no pueden explicar.
5. Similitud significativa entre entregas de distintos grupos.

Principio de responsabilidad académica El aprendizaje no puede delegarse en una herramienta tecnológica. Cuando la Inteligencia Artificial resuelve un problema en lugar del estudiante, se limita el desarrollo de las habilidades cognitivas que el curso busca fortalecer. En este sentido, el taller no pretende evaluar únicamente que el programa produzca un resultado correcto, sino que el estudiante demuestre comprensión del proceso de diseño, análisis y justificación formal que conduce a dicho resultado.

Consecuencias del uso indebido de IA: el uso indebido de IA podrá implicar:

1. Anulación del ejercicio.
2. Anulación del taller completo.
3. Reporte por falta a la ética académica según reglamento institucional.

Aclaraciones

1. El taller es en grupos de tres a cuatro estudiantes.
2. Se debe subir al campus virtual en el enlace correspondiente a este taller un archivo comprimido **.zip** que siga la convención *Código de Estudiante 1-Código de Estudiante 2-Código de Estudiante 3...-Taller 2FLP.zip*.
Este archivo debe contener los archivos: **ejercicio1.rkt**, **ejercicio2.rkt**, **ejercicio3.rkt** que contengan el desarrollo de los ejercicios.
3. Para todos los tipos de datos se debe **incluir la gramática que utilizó**.
4. En las primeras líneas de cada archivo deben estar comentados los nombres y los códigos de los estudiantes participantes.
5. Se debe incluir para cada procedimiento un comentario que explique lo que realiza cada función y para qué es empleada. También deben documentar los procedimientos que hayan implementado como solución a los problemas y de igual manera las funciones auxiliares que se utilicen, con ejemplos de prueba (mínimo 2 pruebas).

Entregas Tardías o por Otros Medios

1. Este taller **sólo se recibirá a través del campus virtual**. Adicionalmente, sólo se evaluarán los documentos solicitados en el punto 2 de la sección anterior. Cualquier otro tipo de correo o nota aclaratoria será descartado.
2. Las entregas tarde serán penalizadas así: (-1pt de la nota final obtenida) por cada hora de retraso o fracción. Por ejemplo, si usted realiza su entrega y el campus registra las 24:00 (i.e., 1min después de la hora de entrega), usted está incurriendo en la primer hora de retraso. Asegurese con mínimo dos horas de anticipación que el link de carga funciona correctamente toda vez que es posible incurrir en una entrega tardía debido a los tiempos de respuesta.